Five-minute pitch — script and shot list

Every case id, number and command below was run and verified. Timings are measured, not estimated. Nothing here claims a figure the evaluation report does not contain.

Before you record

Run this once. Takes about two minutes end to end.

```
cd d:/Projects/Razorpay

npm test                          # 110 pass · ~24s
python -m pytest data/ evaluation/ -q   # 39 pass · ~2s

npm run detect                    # 1,500 cases decided · ~26s
npm run fail-demo                 # scripts 3 API failures · ~22s
npm run eval                      # regenerates the report · ~30s
```

Then start both services in **separate terminals** and leave them running:

```
npm run api                       # terminal 1 — API on :3000
npm run web                       # terminal 2 — dashboard on :5173
```

Open these four tabs in the browser, in this order. All are deep-linkable, so you can switch instantly on camera without clicking through:

Tab	URL	Used at
1	`http://localhost:5173/`	0:00, 4:40
2	`http://localhost:5173/#/case/rcv_04532`	1:10
3	`http://localhost:5173/#/case/rcv_00803`	2:00
4	`http://localhost:5173/#/case/rcv_00366`	2:35

Have one terminal visible for the 3:10 beat.

Screen setup: 1440×900 or larger, browser zoom 100%, dark room, terminal font at 16pt or above so text is readable after compression.

0:00–0:35 · The problem

Screen: Tab 1 — Overview, top row visible.

> "When a payment fails, most recovery systems do one of two things. They retry blindly, or they send a reminder to everyone. Both waste money and annoy customers — and afterwards, nobody can tell you why any particular customer was contacted." "This is fifteen hundred failed payments. One point one three crore at risk."

Pause on the four stat cards. Do not scroll yet.

0:35–1:10 · What Reclaim does

Screen: Tab 1 — scroll slowly to the revenue-at-risk chart.

> "Reclaim diagnoses why each payment failed, decides whether recovery is worth attempting, picks one bounded action, and stops when continuing would harm the customer or the merchant."

Point at the grey bars on the right of the chart.

> "The grey bars are causes Reclaim never contacts a customer about. Fraud signals, and merchant-side misconfiguration the customer cannot fix. Not contacting is a decision too, and it is one this system makes deliberately."

1:10–2:00 · One case, start to finish

Screen: Tab 2 — `#/case/rcv_04532`

> "A seventeen thousand rupee payment failed. Gateway code, issuer unavailable."

Point at the left panel.

> "Diagnosed as a temporary bank or network failure, confidence zero point nine nine. That is a deterministic lookup, not a guess — a gateway code is a fact. The model estimates recovery probability separately."

Move to the right panel.

> "All ten policy checks ran, and all ten are shown — not just the ones that blocked. The engine never stops at the first failure, because a merchant asking 'why wasn't this customer contacted?' deserves the whole picture, not whichever rule happened to fire first."

Scroll to the timeline.

> "Detection, diagnosis, the action chosen, and the payment link issued. The action was a fresh link after thirty minutes — long enough for a transient failure to clear, short enough that intent hasn't decayed."

2:00–2:35 · A guardrail refusing money

Screen: Tab 3 — `#/case/rcv_00803`

> "Now the case I find most convincing. Ninety-six thousand rupees. Diagnosed as a temporary bank failure — the most recoverable category there is."

Point at G1 in the policy panel, and the "opted out" badge on the left.

> "And Reclaim will not touch it. The customer opted out. There is no override for that — not for a high-value order, not with merchant approval. The rule has no exception anywhere in the system." "A system that would contact this customer is a system a merchant cannot trust with the other fourteen hundred."

2:35–3:10 · When the payment API fails

Screen: Tab 4 — `#/case/rcv_00366`

> "This is what happens when Razorpay's API fails. Gateway timeout."

Point at the timeline, the "action failed" then "escalated" entries.

> "Reclaim retried exactly once, under the same idempotency key. That matters — if the first call had actually succeeded and we simply lost the response, the provider returns the existing link instead of creating a second one. It failed again, so the case escalated to a human."

Point at the actions table.

> "Two attempts. No payment link. And the audit trail says it explicitly: no customer was contacted. That is the property this system is built to guarantee — an API failure must never mean somebody gets two messages."

3:10–3:25 · Determinism

Screen: Terminal.

```
npm run verify-replay
```

Runs in about nine seconds. Let it finish on camera.

> "Every decision is stored with the exact inputs that produced it. All fifteen hundred replay byte-identically. The policy engine is a pure function — no clock, no database, no network — which is what makes that possible."

3:25–4:25 · Measured results

Screen: Tab 1 → Evaluation tab.

> "Two tiers, and the split is the whole point."

Point at the verified section.

> "Verified: zero policy violations across fifteen hundred cases and six rules. Measured against ground truth, true regardless of what any customer would have done."

Point at the amber caveat box.

> "Overall diagnosis accuracy is ninety-nine point eight percent, and I want to be direct: that is not a result. My generator writes a gateway code and my engine maps that same code back through the same table. That's arithmetic, not inference. The number worth defending is ninety-two and a half percent, on the forty cases where the code is unrecognised and the engine actually has to guess. Small sample, and I say so on the page."

Scroll to the simulated section.

> "Recovery outcomes cannot be measured on synthetic data. Whether a customer would have paid anyway is a counterfactual my generator writes, not something the system discovers. So there is a randomised control arm that receives no contact, and three sensitivity scenarios. Uplift runs between seventeen and forty-one points, and the confidence intervals don't overlap in any of them."

Scroll to the baseline comparison.

> "And here is the number that doesn't flatter me. Contacting everyone recovers thirteen percent more revenue than Reclaim — by sending nineteen percent more messages. That's a trade-off, not a defeat. The floor that produces it is one configurable threshold. What Reclaim gives a merchant is the ability to make that choice deliberately and see exactly what it costs."

4:25–4:45 · Architecture

Screen: README architecture diagram (GitHub or editor).

> "Python trains the model and runs the evaluation. TypeScript makes every decision. The policy engine is pure, which is what makes replay work." "No Redis — a Postgres table with a run-after column gives durable delayed jobs in fifty lines. No Python service in the request path — a logistic regression is a dot product, so the coefficients are exported as JSON and scored in TypeScript, with a parity test asserting it matches scikit-learn to six decimal places. And no language model anywhere near the money path."

4:45–5:00 · Close

Screen: Tab 1 — Overview, back on the zero-violations card.

> "Reclaim doesn't just find lost revenue. It decides what is safe to do about it, proves it never crossed a line, and is honest about which of its numbers are measurements and which are simulations."

Hold on the card for two seconds. End.

If you overrun

Cut in this order — each is self-contained:

1. **3:10 determinism** (15s) — the claim is on the Overview card anyway
2. **4:25 architecture** (20s) — it is in the README a reviewer can read
3. **0:35 what-it-does** (35s) — the case walkthrough demonstrates it implicitly

Never cut 2:00 (the guardrail refusing money) or the caveat at 3:25. Those two beats are the reason this project is worth a panel's time.

Recovery if something breaks mid-take

Symptom	Fix
Dashboard shows "loading…" forever	API died. Restart `npm run api`, wait 5s, reload
Case shows no actions	`npm run fail-demo` was not run — rerun it, reload
Evaluation tab empty	`npm run eval`
Numbers differ from this script	`npm run detect && npm run fail-demo && npm run eval`

The case ids are stable across reruns — the arm assignment is a hash of the case id, not a random draw, so the same cases stay in the same experimental arms every time.